Unit - 3

1D Array in C

A one-dimensional (1D) array is a collection of elements of the same data type stored in contiguous memory locations. Each element is accessed using an index.

Declaration of 1D Array

Array declaration specifies the data type, array name, and number of elements.

Syntax: data_type array_name[size];

Example: int marks[5];

This declares an integer array named marks that can store five elements.

Array indexing:

Element	Index
First element	0
Second element	1
Third element	2
Fourth element	3
Fifth element	4

In C, array indexing starts from 0 and ends at size - 1.

Input in 1D Array

The scanf() function is used to take input for array elements.

Example:

```
int a[5], i;
for (i = 0; i < 5; i++)
{
   scanf("%d", &a[i]);
}
```

The loop takes input for all five elements, one by one.

Output of 1D Array

The printf() function is used to display array elements.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
int i;
for (i = 0; i < 5; i++)
{
   printf("%d ", a[i]);
}
```

Output: 10 20 30 40 50

Traversal of 1D Array

Array traversal is the process of visiting and accessing each element of an array one by one.

A for loop is commonly used for traversal.

Syntax:

```
for (i = 0; i < size; i++)
{
   // Process array element
}
```

Example:

```
#include <stdio.h>
int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int i;
   for (i = 0; i < 5; i++)
   {
      printf("a[%d] = %d\n", i, a[i]);
   }
   return 0;
}
```

Output:

a[0] = 10

a[1] = 20

a[2] = 30

a[3] = 40

a[4] = 50

Complete Program: Input, Output and Traversal

```
#include <stdio.h>
int main()
{
   int a[5], i;
   printf("Enter 5 elements:\n");
   for (i = 0; i < 5; i++)
   {
      scanf("%d", &a[i]);
   }
   printf("Array elements are:\n");
   for (i = 0; i < 5; i++)
   {
      printf("%d ", a[i]);
   }
   return 0;
}
```

Input:

10 20 30 40 50

Output:

Array elements are:

10 20 30 40 50

1. Find Maximum Element in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 25, 5, 40, 15};

int i, max = a[0];

for (i = 1; i < 5; i++)

{

if (a[i] > max)

max = a[i];

}

printf("Maximum = %d", max);

return 0;

}
```

Output:

Maximum = 40

2. Find Minimum Element in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 25, 5, 40, 15};

int i, min = a[0];

for (i = 1; i < 5; i++)

{
```

```
if (a[i] < min)

min = a[i];

}

printf("Minimum = %d", min);

return 0;

}
```

Output:

Minimum = 5

3. Find Sum of Elements in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 20, 30, 40, 50};

int i, sum = 0;

for (i = 0; i < 5; i++)

{

sum = sum + a[i];

}

printf("Sum = %d", sum);

return 0;

}
```

Output:

Sum = 150

4. Find Average of Elements in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 20, 30, 40, 50};

int i, sum = 0;

float avg;

for (i = 0; i < 5; i++)

{

sum = sum + a[i];

}

avg = (float)sum / 5;

printf("Average = %.2f", avg);

return 0;

}
```

Output:

Average = 30.00

Strings in C

A string in C is a sequence of characters terminated by the null character '\0'. Strings are stored in character arrays.

Example:

```
char str[20] = "Hello";
```

Header file required for string functions:

```
#include <string.h>
```

1. gets() and puts()

gets() is used to read a string from the keyboard, including spaces. However, it is unsafe because it does not check the array size and has been removed from the C11 standard. Use fgets() instead.

Syntax:

fgets(str, sizeof(str), stdin);

puts() is used to display a string on the screen and automatically adds a newline.

Syntax:

puts(str);

Example:

```
#include <stdio.h>

int main()

{

   char str[50];

   printf("Enter a string: ");

   fgets(str, sizeof(str), stdin);

   puts(str);

   return 0;

}
```

Output:

Enter a string: Hello World

Hello World

2. strlen()

The strlen() function returns the length of a string, excluding the null character '\0'.

Syntax: strlen(string);

Example:

```
#include <stdio.h>

#include <string.h>

int main()

{

   char str[] = "Hello";

   printf("Length = %d", (int)strlen(str));

   return 0;

}
```

Output:

Length = 5

3. strcpy()

The strcpy() function copies the source string into the destination string, including the terminating null character.

Syntax:

```
strcpy(destination, source);
```

Example:

```
#include <stdio.h>

#include <string.h>

int main()

{

   char source[] = "Hello";

   char destination[20];

   strcpy(destination, source);

   printf("%s", destination);
```

```
    return 0;
}
```

Output:

Hello

4. strcmp()

The strcmp() function compares two strings lexicographically, character by character.

Syntax:

strcmp(string1, string2);

Return values:

- 0 — both strings are equal.
- Negative value — the first string is smaller.
- Positive value — the first string is greater.

Example:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[] = "Hello";
    char str2[] = "Hello";
    if (strcmp(str1, str2) == 0)
        printf("Strings are equal");
    else
        printf("Strings are not equal");
    return 0;
```

```
}
```

Output:

Strings are equal

Difference Between String Functions

Function	Purpose
fgets()	Reads a string safely with a specified size limit.
puts()	Displays a string followed by a newline.
strlen()	Calculates the length of a string.
strcpy()	Copies one string into another.
strcmp()	Compares two strings.